DATA STRUCTURES & ALGORITHMS LAB

ASSIGNMENT 2

Complexity Comparison

Student Name	**Shailja Rathore**
Roll No.	**2401201060**
Course / Semester	**BCA(AI&DS) - Section A**
Subject	**Design and Analysis of Algorithms**

Algorithm Covered

- Bubble Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Fibonacci – Recursive
- Fibonacci – Iterative
- Fibonacci – Dynamic Programming

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
Fibonacci – Recursive	$O(2^n)$	$O(2^n)$	$O(2^n)$	O(n)
Fibonacci – Iterative	O(n)	O(n)	O(n)	O(1)
Fibonacci – Dynamic Programming	O(n)	O(n)	O(n)	O(n)

Explanation of Complexity Table

Bubble Sort: The best-case complexity is O(n) when the input is already sorted and the algorithm uses early termination. For average and worst-case inputs, repeated comparisons and swaps result in $O(n^2)$ time. It requires O(1) auxiliary space.

Insertion Sort: Insertion Sort has a best-case time complexity of O(n) when the data is already sorted or nearly sorted. In average and worst cases, elements may need to be shifted repeatedly, resulting in $O(n^2)$ time. Its auxiliary space complexity is O(1).

Merge Sort: Merge Sort has O(n log n) time complexity in the best, average, and worst cases because the array is consistently divided and merged. It provides predictable performance for large inputs but requires O(n) auxiliary space for the merging process.

Quick Sort: Quick Sort has O(n log n) best-case and average-case time complexity when the partitions are reasonably balanced. Its worst-case complexity is O(n), according to the recursion-stack space listed in the assignment table, while the table specifies O(log n) space complexity. With the required last-element pivot, however, sorted or reverse-sorted inputs can produce highly unbalanced partitions, which is important when interpreting the experimental results.

Fibonacci – Recursive: The recursive Fibonacci approach has $O(2^n)$ time complexity in the best, average, and worst cases because it repeatedly recalculates the same Fibonacci values. The recursive calls require O(n) stack space.

Fibonacci – Iterative: The iterative Fibonacci algorithm calculates the sequence using a single loop, giving O(n) time complexity in all cases. Since it maintains only a small number of variables, its auxiliary space complexity is O(1).

Fibonacci – Dynamic Programming: Dynamic Programming improves Fibonacci computation by storing previously calculated values. This gives O(n) time complexity while requiring O(n) memory to store the computed Fibonacci values.

Overall Observation

The table shows that algorithms with quadratic or exponential time complexity become increasingly expensive as the input size grows. Merge Sort provides consistent O(n log n) performance, while Bubble Sort and Insertion Sort can require $O(n^2)$ time. Among the Fibonacci approaches, Recursive Fibonacci is the least efficient because of repeated calculations, whereas Iterative and Dynamic Programming approaches reduce the running time to O(n).